

Bizu do Salles: guia de comandos, banco, segurança e hospedagem

Um mapa para você **abrir, conferir, adicionar e remover** cada parte do projeto no VS Code (Windows).
Guarde este PDF junto do `docs/RELATORIO.pdf`, que explica o projeto em detalhe.

Sumário

1. Abrir o projeto e o terminal
2. Comandos do dia a dia
3. Mapa: onde está cada coisa
4. Adicionar e remover
5. Conferir se está tudo certo
6. Erros comuns
7. Banco de dados: acessar, ver e editar
8. Pagamentos, acesso dos alunos e o seu dinheiro
9. Segurança: como se proteger
10. Deixar o site online 24 horas de graça
11. Servidor em casa: peças, links e montagem

Regra de ouro: antes de mexer, rode `git pull`. Depois de mexer, rode `npm test`. Se passar, está seguro.

1. Abrir o projeto e o terminal

1. No VS Code: **Arquivo** → **Abrir Pasta...** e escolha `WillCoder01\PROJETOS\bizu-do-salles`.
2. Abra o terminal: menu **Terminal** → **Novo Terminal** (ou `Ctrl + ``).
3. Se o PowerShell bloquear scripts, clique na setinha ao lado do **+** do terminal e escolha **Command Prompt**.

Atalho do VS Code	Para quê
<code>Ctrl + P</code>	Achar um arquivo pelo nome (digite, por exemplo, <code>site.ts</code>).
<code>Ctrl + Shift + F</code>	Procurar um texto em todo o projeto (por exemplo, uma frase da página inicial).
<code>Ctrl + F</code>	Procurar dentro do arquivo aberto.
<code>Ctrl + S</code>	Salvar.
<code>Ctrl + Z</code>	Desfazer.
<code>Ctrl + C</code> (no terminal)	Parar o site que está rodando.

2. Comandos do dia a dia

Comando	O que faz	Quando usar
<code>npm install</code>	Baixa as bibliotecas e gera o Prisma.	Na primeira vez e depois de cada <code>git pull</code> que mude o <code>package.json</code> .
<code>npm run dev</code>	Liga o site em http://localhost:3000 .	Para ver e testar. A tela atualiza sozinha quando você salva.
<code>npm test</code>	Confere regras, questões e conteúdo.	Sempre depois de editar.
<code>npm run build</code>	Monta a versão final, como no servidor.	Antes de publicar.
<code>npm run db:seed</code>	Carrega no banco as questões e os materiais novos dos arquivos JSON.	Depois de adicionar arquivos em <code>content/</code> .
<code>npx prisma migrate deploy</code>	Cria ou atualiza as tabelas do banco.	Na primeira vez e quando vier migração nova.
<code>npm run admin:create</code> <code>-- email@x.com "Nome"</code>	Cria um usuário administrador.	Uma vez, ou para criar outro admin.
<code>npx prisma studio</code>	Abre o banco no navegador, como uma planilha.	Para conferir dados. Evite apagar por lá.
<code>npm run content:report</code>	Mostra quantas questões existem por disciplina e por gabarito.	Para ver o tamanho do banco de questões.
<code>npm run relatorio</code>	Gera de novo o <code>docs/RELATORIO.pdf</code> .	Depois de mudar a documentação.

Git: salvar e baixar alterações

<code>git pull</code>	← baixa o que foi mergeado no GitHub
<code>git status</code>	← mostra o que você mudou
<code>git add .</code>	← marca tudo para salvar
<code>git commit -m "o que eu fiz"</code>	← salva uma versão
<code>git push</code>	← envia para o GitHub

Desfazer mudanças de um arquivo que você ainda **não** commitou: `git restore caminho/do/arquivo`.

3. Mapa: onde está cada coisa

Quero mudar...	Arquivo ou lugar
Nome do site, textos da página inicial, cartões e perguntas frequentes	<code>src/config/site.ts</code>
Prazos, limites de tentativas, simulado, ranking, Raio-X e questões grátis	<code>src/config/regras.ts</code>
Cores e visual (claro e escuro)	<code>src/app/globals.css</code> (variáveis <code>--brand</code> , <code>--accent</code> e <code>--bg</code> no topo)
Nomes "Ciclo Básico" e "Fácil/Média/Difícil"	<code>src/lib/format.ts</code>
Menu do aluno	<code>src/app/app/layout.tsx</code> (lista <code>NAV</code>)
Menu do admin	<code>src/app/admin/layout.tsx</code>
Termos de uso e privacidade	<code>src/app/termos/page.tsx</code> e <code>src/app/privacidade/page.tsx</code>
Questões em lote	<code>content/questoes/*.json</code>
Resumos e materiais	<code>content/materiais/*.json</code>
Senhas, banco e chaves (Neon, Mercado Pago, e-mail)	<code>.env</code> (nunca envie para o GitHub)
Preços, planos, cupons, usuários e escolas	Painel do admin no site: <code>/admin</code>
Estrutura do banco (tabelas)	<code>prisma/schema.prisma</code> (só com ajuda: exige migração)

Cada arquivo importante começa com um cabeçalho  **O QUE É /**  **EDITÁVEL /**  **CUIDADO.** Para achar todos os pontos editáveis, use `Ctrl + Shift + F` e busque .

Pastas principais

```
bizu-do-salles/
├─ src/app/           ← as páginas (cada pasta vira um endereço)
│   ├─ page.tsx       ← página inicial (/)
│   └─ app/           ← área do aluno (/app/...)
│       └─ admin/      ← painel do admin (/admin/...)
├─ src/config/         ←  textos e regras (comece por aqui)
├─ src/lib, src/core   ← lógica e segurança (não mexa sem ajuda)
├─ content/           ←  questões e materiais em JSON
├─ prisma/            ← banco de dados e carga inicial
├─ tests/, e2e/       ← testes automáticos
└─ docs/              ← guias e o RELATORIO.pdf
```

4. Adicionar e remover: passo a passo

4.1 Questões

Uma questão (mais fácil): entre no site como admin, vá em **Admin** → **Questões** → **Nova questão**, preencha e salve.

Várias questões (arquivo):

1. Em `content/questoes/`, copie um arquivo parecido e renomeie (por exemplo, `sp-cesp-01-seguranca.json`).
2. Troque o conteúdo mantendo o formato abaixo. O `code` de cada questão precisa ser único.
3. Rode `npm test` e depois `npm run db:seed`.

```
{
  "cycle": "SPECIFIC",           ← "BASIC" ou "SPECIFIC"
  "subject": "Disciplina",
  "topic": "Assunto",
  "state": "SP",                 ← apague esta linha para valer no Brasil
  "todo":
    "sourceLicense": "Questões originais Bizu do Salles.",
    "author": "Bizu do Salles",
    "questions": [
      {
        "code": "CESP-SEG-001",
        "difficulty": "EASY",     ← EASY, MEDIUM ou HARD
        "statement": "Enunciado...",
        "options": { "A": "...", "B": "...", "C": "...", "D": "...", "E": "..." },
        "correct": "C",
        "explanation": "Por que a C está certa (mínimo 20 letras).",
        "reference": "Art. 5º, inciso X"
      }
    ]
}
```

Também dá para importar CSV: **Admin** → **Importar**.

Remover uma questão: em **Admin** → **Questões**, abra a questão e mude o status para **Arquivada**. Assim ela some para os alunos, mas o histórico deles continua certo.

Apagar do JSON **não** tira a questão do banco: o seed só adiciona. Remova sempre pelo painel.

4.2 Materiais e resumos

Pelo painel: **Admin** → **Materiais**, para criar, editar ou despublicar.

Por arquivo: copie um arquivo de `content/materiais/`, troque `title` e `body` (aceita `##` títulos, `**negrito**` e listas com `-`) e rode `npm run db:seed`. Com `"free": true`, o material fica liberado para todos.

4.3 Planos, preços e cupons

Tudo pelo painel: **Admin** → **Planos e preços** e **Admin** → **Cupons**. Não precisa mexer em código.

4.4 Usuários e acesso

Admin → **Usuários**: liberar ou estender um plano, bloquear, trocar o papel (ADMIN/EDITOR). Para criar um admin, use `npm run admin:create`.

4.5 Textos, cartões e perguntas frequentes

Abra `src/config/site.ts`.

- **Adicionar um cartão:** copie uma linha `{ icone: ..., titulo: ..., texto: ... }`, e cole logo abaixo.
- **Adicionar uma pergunta frequente:** copie uma linha `["pergunta", "resposta"],`.
- **Remover:** apague a linha inteira, incluindo a vírgula do fim.

4.6 Item do menu do aluno

Em `src/app/app/layout.tsx`, na lista `NAV`: cada linha é um item. **Cuidado:** a lista `MOBILE`, logo abaixo, usa as posições da `NAV` (`NAV[0]`, `NAV[9]` ...); se apagar uma linha, confira essas posições. Para acrescentar, a página precisa existir em `src/app/app/NOME/page.tsx`. Peça ajuda nesse caso.

4.7 Regras e números

Em `src/config/regras.ts`, troque só os números. Exemplos: `diasValidade: 30` define quantos dias o aluno fica logado; `gratisPorArquivo: 3` define quantas questões grátis cada arquivo novo libera.

5. Conferir se está tudo certo

Verificação	Como
O site abre?	<code>npm run dev</code> e abra <code>http://localhost:3000</code>
Minhas edições estão válidas?	<code>npm test</code> (tudo verde = ok)
Vai funcionar no servidor?	<code>npm run build</code>
O banco tem os dados?	<code>npx prisma studio</code>
Quantas questões tenho?	<code>npm run content:report</code>
O que eu mudei?	<code>git status</code> e <code>git diff</code>

6. Erros comuns

Mensagem	Solução
"execução de scripts foi desabilitada"	Use o Command Prompt ou rode <code>Set-ExecutionPolicy -Scope CurrentUser RemoteSigned</code>
<code>EPERM</code> ou <code>ENOTEMPTY</code> no <code>npm install</code>	Feche o VS Code e o site, rode <code>rmdir /s /q node_modules</code> e depois <code>npm install</code>
"@prisma/client did not initialize yet"	<code>npx prisma generate</code>
"Port 3000 is in use"	Já existe um site rodando: feche o outro terminal ou use <code>Ctrl + C</code>
Erro no <code>npm test</code> apontando um JSON	Vírgula faltando ou sobrando, ou <code>code</code> repetido. O erro mostra o arquivo e a linha.
Página em branco	Pare com <code>Ctrl + C</code> , rode <code>git pull</code> e <code>npm install</code> , depois <code>npm run dev</code> de novo.

7. Banco de dados: acessar, ver e editar

O banco é um **PostgreSQL** hospedado no **Neon**. O endereço e a senha dele ficam no arquivo `.env`, nas linhas `DATABASE_URL` e `DATABASE_URL_POOLED`.

Três formas de acessar (da mais segura para a mais arriscada)

Forma	Como	Use para
1. Painel do admin (recomendado)	Entre no site com a sua conta admin e acesse <code>/admin</code>	Tudo do dia a dia: questões, materiais, planos, cupons, usuários, escolas. Fica registrado na Auditoria .
2. Prisma Studio	No terminal: <code>npx prisma studio</code> . Abre <code>http://localhost:5555</code> com as tabelas em formato de planilha.	Conferir dados, corrigir um campo pontual.
3. Console do Neon	console.neon.tech → seu projeto → Tables (planilha) ou SQL Editor (comandos)	Ver o banco de qualquer lugar, criar backups (branches), trocar a senha.

Tabelas principais

Tabela	O que guarda
<code>User</code>	Alunos e admins (e-mail, nome, papel, estado). A senha fica embaralhada (hash); ninguém consegue lê-la.
<code>Plan</code>	Planos à venda (nome, preço em centavos, dias de acesso, ciclos).
<code>Subscription</code>	O acesso de cada aluno: <code>status</code> (PENDING, ACTIVE, CANCELED, EXPIRED) e <code>endsAt</code> (até quando vale).
<code>Payment</code>	Cada pagamento recebido do Mercado Pago (valor, status, data).
<code>Coupon</code>	Cupons de desconto.
<code>Question</code> e <code>QuestionOption</code>	Questões e alternativas.
<code>QuestionAttempt</code>	Cada resposta de cada aluno (base do Raio-X e do ranking).
<code>Material</code>	Resumos e PDFs.
<code>AuditLog</code>	Histórico do que os admins fizeram. Nunca apague.

Editar ou acrescentar pelo Prisma Studio

- Rode `npx prisma studio` e abra `http://localhost:5555`.
- Clique na tabela (por exemplo, `Subscription`).
- Editar:** dê dois cliques na célula, mude o valor e clique em **Save 1 change**.
- Acrescentar:** botão **Add record**, preencha e salve.
- Remover:** marque a linha e clique em **Delete**. Evite fazer isso: prefira desativar ou arquivar pelo painel.

Antes de editar direto no banco: faça um backup. No Neon, crie um **Branch** (uma cópia instantânea), ou rode `npm run db:backup` (precisa do `pg_dump` ; no Windows, use o Branch do Neon). Nunca edite `AuditLog` , `Payment` nem a senha (`passwordHash`) de ninguém.

8. Pagamentos, acesso dos alunos e o seu dinheiro

O site **não guarda dinheiro nem "saldo"** de aluno. Funciona assim:

1. O aluno escolhe um plano e paga no **Mercado Pago** (Pix ou cartão).
2. O Mercado Pago avisa o site (webhook). O site confere a assinatura do aviso e o valor, e grava um `Payment APPROVED`.
3. O acesso (`Subscription`) fica **ACTIVE** até a data `endsAt` .
4. O **dinheiro** cai na **sua conta do Mercado Pago**.

Quero...	Onde
Ver quanto recebi, sacar ou transferir	App ou site do Mercado Pago → Seu dinheiro / Atividade . É lá que fica o seu saldo real.
Ver quem pagou	Painel <code>/admin</code> (visão geral) ou a tabela <code>Payment</code> no Prisma Studio
Liberar acesso de quem pagou por fora (Pix direto)	Admin → Usuários → Pedidos aguardando pagamento → Confirmar pagamento e liberar . Só faça isso depois de ver o dinheiro no extrato.
Dar mais dias a um aluno	Admin → Usuários : liberar ou estender o plano. Fica registrado na auditoria.
Devolver dinheiro (reembolso)	No Mercado Pago, na venda → Devolver dinheiro . Depois bloqueie ou encerre o acesso no painel.
Mudar preços ou criar cupom	Admin → Planos e preços e Admin → Cupons
Ligar o Mercado Pago	No <code>.env</code> : <code>MERCADOPAGO_ACCESS_TOKEN</code> e <code>MERCADOPAGO_WEBHOOK_SECRET</code> (veja <code>docs/PAYMENTS.md</code>)

Não mude `status` nem `endsAt` na mão pelo banco se puder fazer pelo painel: pelo painel fica registrado quem mudou e quando.

9. Segurança: como se proteger

O que o site já faz sozinho: senhas com Argon2id (não dá para descobrir), um aparelho por conta, limite de tentativas de login, verificação de acesso no servidor, cabeçalhos de segurança (CSP), conferência de assinatura e valor nos pagamentos, e auditoria das ações do admin.

O que depende de você: checklist

✓	Ação	Como
☐	Trocar a senha do banco (ela apareceu em um print)	Neon → Roles → <code>neondb_owner</code> → Reset password . Depois atualize <code>DATABASE_URL</code> e <code>DATABASE_URL_POOLED</code> no <code>.env</code> (e na Vercel, se já tiver publicado).
☐	Nunca enviar o <code>.env</code> para o GitHub nem mostrar em print	Ele já está no <code>.gitignore</code> . Antes de mandar um print, esconda senhas e chaves.
☐	Senha forte e única no admin	Com 14 ou mais caracteres. Use um gerenciador (Bitwarden, gratuito).
☐	Ativar a verificação em duas etapas (2FA)	No GitHub, Neon, Vercel, Mercado Pago e no seu e-mail.
☐	Poucos admins	Dê o papel EDITOR para quem só cuida de questões.
☐	Backups	<code>npm run db:backup</code> toda semana. O Neon também guarda um histórico curto.
☐	Atualizar as bibliotecas	Uma vez por mês: <code>npm audit</code> . Peça ajuda para atualizar e rode <code>npm test</code> .
☐	Revisar a auditoria	Admin → Auditoria : veja se há ações que você não reconhece.
☐	Usar só HTTPS	A Vercel e o Cloudflare já dão HTTPS grátis.
☐	Proteção extra contra ataques	Coloque o domínio no Cloudflare (plano grátis) : esconde o servidor e barra robôs.
☐	Termos e privacidade (LGPD)	Revise <code>/termos</code> e <code>/privacidade</code> com um advogado antes de vender.

Se desconfiar de invasão: 1) troque a senha do banco e do admin; 2) troque as chaves do Mercado Pago; 3) veja **Admin** → **Auditoria**; 4) restaure o backup se algum dado sumiu.

10. Deixar o site online 24 horas de graça

Recomendação: Vercel (site) + **Neon** (banco). As duas têm plano gratuito, ficam no ar 24 horas, dão HTTPS e não dependem da sua luz ou internet. É o caminho mais seguro para começar.

1. Crie a conta em vercel.com/signup entrando com o GitHub.
2. **Add New** → **Project** → escolha o repositório `WillCoder01`.
3. Em **Root Directory**, coloque `PROJETOS/bizu-do-salles`.
4. Em **Environment Variables**, copie as linhas do seu `.env` (`DATABASE_URL`, `DATABASE_URL_POOLED`, `APP_URL`, as chaves do Mercado Pago...). `APP_URL` passa a ser o endereço da Vercel.
5. **Deploy**. Em cerca de 2 minutos, o site estará em `https://seu-projeto.vercel.app`.
6. A cada merge no GitHub, a Vercel publica sozinha.
7. (Opcional) Domínio próprio (`bizudosalles.com.br`, cerca de R\$ 40 por ano em registro.br):
Vercel → **Settings** → **Domains**.

Passo a passo completo em `docs/DEPLOY.md`.

Opção	Custo	Prós	Contras
Vercel + Neon ★	R\$ 0	Zero manutenção, rápido, HTTPS, não cai com a sua luz	Limites do plano grátis (sobram para começar). O plano grátis da Vercel é para uso pessoal: quando começar a vender, avalie o plano Pro.
Servidor em casa + Cloudflare Tunnel	Peças + cerca de R\$ 5 a 15 por mês de luz	Controle total, aprendizado	Cai se faltar luz ou internet; você cuida das atualizações e da segurança
Oracle Cloud (Always Free)	R\$ 0	Servidor Linux real e grátis	Cadastro pede cartão; é mais técnico

11. Servidor em casa: peças, links e montagem

Se quiser mesmo ter o seu servidor, abaixo estão duas configurações. Os links levam para o site oficial ou para uma busca no Mercado Livre: **confira o vendedor, a reputação e o preço atual** antes de comprar. Os preços são estimativas.

Opção A: econômica (Raspberry Pi 5), cerca de R\$ 900 a 1.300

Peça	Por quê	Link
Raspberry Pi 5, 8 GB	Computador pequeno que gasta pouca luz (cerca de 5 a 10 W)	site oficial · Mercado Livre
Fonte oficial 27 W USB-C	Fonte fraca trava o Pi	oficial · ML
Case com cooler ativo	Evita superaquecimento 24 horas por dia	oficial · ML
SSD NVMe 256 GB + adaptador (HAT) M.2	Muito mais durável e rápido que o cartão SD	HAT oficial · SSD no ML
Cartão microSD 32 GB	Só para instalar o sistema	ML
Cabo de rede (Ethernet)	Mais estável que Wi-Fi	ML
Nobreak 600 VA	Segura quedas curtas de energia	ML

Opção B: mais forte (mini PC), cerca de R\$ 1.200 a 2.000

Peça	Por quê	Link
Mini PC Intel N100, 16 GB de RAM, SSD de 512 GB	Mais potente que o Pi, gasta cerca de 10 a 15 W e já vem montado	Mercado Livre
Nobreak 600 VA	Segura quedas de energia	ML
Cabo de rede	Estabilidade	ML

Um notebook velho com 8 GB de RAM também serve para testar, e custa R\$ 0.

Montagem do software (resumo)

1. Instale o sistema: **Raspberry Pi OS Lite 64-bit** (com o [Raspberry Pi Imager](#)) ou **Ubuntu Server 24.04** no mini PC.
2. Instale o Node.js 22 e o git; clone o repositório; crie o `.env`.
3. Banco: continue usando o Neon (mais fácil) ou instale o PostgreSQL 16 no próprio servidor.
4. `npm install` → `npx prisma migrate deploy` → `npm run build`.
5. Deixe rodando sempre com o **PM2**: `npm i -g pm2`, depois `pm2 start "npm start" --name bizu`, depois `pm2 startup` e `pm2 save`. Assim o site religa sozinho se o servidor reiniciar.
6. Coloque na internet **sem abrir portas no roteador** com o [Cloudflare Tunnel](#) (grátis). Ele dá HTTPS, esconde o IP da sua casa e protege contra ataques. É preciso ter um domínio no Cloudflare.
7. Segurança do servidor: atualizações automáticas (`unattended-upgrades`), firewall (`ufw enable`), login SSH só com chave e backup diário do banco.

Minha sugestão: publique agora na **Vercel + Neon** (grátis e estável). Monte o servidor em casa como projeto de aprendizado e use-o como **ambiente de testes**. Se um dia ele estiver bem estável, você decide migrar.

Resumo: textos em `src/config/site.ts` · regras em `src/config/regras.ts` · cores em `globals.css` · questões e materiais em `content/` ou no painel · preços, cupons e usuários no painel · depois de editar, sempre `npm test`.